Property Insurance System

API Documentation · March 2026

Customer | Agent | Claims Officer | Admin

Base URL: http://localhost:<port>/api · Authentication: JWT Bearer Token · Content-Type: application/json · Currency: INR

🏠 **Property Insurance System** Overview & Table of Contents

📊 System Overview

Key statistics at a glance

8	34	4	8
CONTROLLERS	ENDPOINTS	USER ROLES	POLICY STEPS

👥 User Roles & Capabilities

Each role has scoped access to specific endpoints

Customer
- Self-registration
- Submit policy requests
- File insurance claims
- View invoices
- Manage notifications

Agent
- View assigned policies
- Send property forms
- Calculate risk scores
- Track commissions
- View approved policies

Claims Officer
- View pending claims
- Approve/Reject claims
- View claim history
- Trigger auto-invoices

Admin
- Create agents & officers
- Assign agents to policies
- Full dashboard stats
- Manage categories & plans
- Final policy approval

📋 Table of Contents

All 8 controllers and their base routes

1	Auth	`/api/auth`	4 endpoints
2	Admin	`/api/admin`	6 endpoints
3	Policy Requests	`/api/policyrequests`	10 endpoints
4	Claims	`/api/claims`	4 endpoints
5	Category	`/api/category`	3 endpoints
6	Property Plans	`/api/propertyplans`	3 endpoints
7	Invoices	`/api/invoices`	1 endpoint
8	Notifications	`/api/notifications`	3 endpoints

🔐 **Authentication:** All protected endpoints require `Authorization: Bearer <jwt_token>` header. Token obtained via `POST /api/auth/login`.

🏠 **Property Insurance System** 1. Auth — 2. Admin

🔐 1. Auth `/api/auth`

Handles registration, login, user creation, and referral redemption

POST `/api/auth/register` ❌ No Auth Public

Register a new customer account. Optional referral code grants ₹50 reward to referrer.

Field	Type	Required	Description
`fullName`	string	✅	Customer's full name
`email`	string	✅	Valid email address
`password`	string	✅	e.g. Secret@123
`role`	string	✅	Must be "Customer"
`referralCode`	string	❌	Optional referral code

✅ 200 OK — Returns JWT token, role, referral code, balance, and expiration

POST `/api/auth/login` ❌ No Auth Public

Authenticate any user and receive a JWT Bearer token for subsequent requests.

REQUEST

```
{
  "email": "admin@gmail.com",
  "password": "Admin@123"
}
```

RESPONSE 200

```
{
  "token": "<jwt>",
  "role": "Admin",
  "referralBalance": 150.00
}
```

POST `/api/auth/create-user` ✅ Auth Admin

Admin creates Agent or ClaimsOfficer accounts. Role must be "Agent" or "ClaimsOfficer" — cannot be "Customer".

POST `/api/auth/redeem` ❌ No Auth Public

Redeem referral balance to wallet. Body: `{ "email": "...", "amount": 50 }`

⚙️ 2. Admin `/api/admin`

All endpoints require Admin role

Method	Endpoint	Description
GET	/api/admin/dashboard	Welcome message for admin
GET	/api/admin/agents	List all active agents
GET	/api/admin/staff	All active users ordered by role
GET	/api/admin/stats	Full dashboard KPI statistics
GET	/api/admin/agent-area	Agent welcome endpoint
GET	/api/admin/customer-area	Returns all property plans for customers

📊 **Admin Stats** returns: totalCustomers, totalAgents, totalClaimsOfficers, totalPolicies, totalClaims, totalRevenue, revenueGrowth, topPlans, and topAgents.

3. Policy Requests `/api/policyrequests`

Manages the complete 8-step policy lifecycle from creation to final approval

1 Create → 2 Pending → 3 Assign Agent → 4 Send Form → 5 Submit Form → 6 Calc Risk → 7 Buy → 8 Approve

POST `/api/policyrequests/create` Step 1 Auth Customer

Customer submits a new policy request. Body: `{ "planId": 1 }`

GET `/api/policyrequests/admin/pending` Step 2 Auth Admin

Admin views all pending/confirmed policy requests with customer and plan details.

PUT `/api/policyrequests/{id}/assign-agent/{agentId}` Step 3 Auth Admin

Admin assigns an agent to a policy request. Optional body: instruction message. Sends notifications to both agent and customer.

GET `/api/policyrequests/agent/assigned` Step 3.1 Auth Agent

Agent views their currently assigned policy requests (statuses: AgentAssigned, FormSent, FormSubmitted, RiskCalculated).

PUT `/api/policyrequests/{id}/send-form` Step 4 Auth Agent

Agent notifies customer to submit their property details. Sends in-app notification.

PUT `/api/policyrequests/{id}/submit-form` Step 5 Auth Customer

Customer submits property details for risk assessment.

Field	Type	Description
`propertyAddress`	string	Full address of the property
`propertyValue`	decimal	Estimated property value in ₹
`propertyAge`	integer	Age of the property in years

PUT `/api/policyrequests/{id}/calculate-risk` Step 6 Auth Agent

Agent calculates risk score, premium, and installments using the formula below.

RISK LOGIC

- PropertyAge > 10 → +30 risk points
- PropertyValue > ₹2,50,000 → +40 risk points
- FinalPremium = BasePremium + (RiskScore × ₹500)
- Quarterly=4 · HalfYearly=2 · Yearly=1

RESPONSE

```
{
  "riskScore": 70,
  "totalPremium": 36250.00,
  "installmentCount": 1,
  "agentCommission": 125.00
}
```

PUT `/{id}/buy` Step 7

Customer

Customer confirms purchase. Notifies all admins for final approval.

PUT `/{id}/admin-approve` Step 8

Admin

Admin gives final approval. Sends congratulatory notification to customer.

Method	Endpoint	Role	Description
GET	/customer/my-requests	Customer	All policy requests for logged-in customer
GET	/agent/approved	Agent	Approved policies for commission tracking

🏠 **Property Insurance System** 4. Claims · 5. Category · 6. Property Plans

🛡️ 4. Claims `/api/claims`

Customer files claims; ClaimsOfficer reviews and approves/rejects

POST `/api/claims` ✅ Auth Customer

Customer files an insurance claim on an approved policy. Sends notifications to all ClaimsOfficers.

Field	Type	Description
`policyRequestId`	integer	Active approved policy ID
`propertyAddress`	string	Property location
`propertyValue`	decimal	Claimed property value
`propertyAge`	integer	Age of the property

Method	Endpoint	Role	Description
GET	`/api/claims/pending`	ClaimsOfficer	All pending claims awaiting review
GET	`/api/claims/history`	ClaimsOfficer	Approved/Rejected claims history
PUT	`/api/claims/{id}/verify`	ClaimsOfficer	Approve or reject a claim

💡 **Auto-Invoice:** When a claim is approved, an invoice is automatically generated and the customer is notified. Rejection also triggers a notification with remarks.

Verify Request Body: `{ "isAccepted": true, "remarks": "Documents verified" }`

📁 5. Category `/api/category`

Hierarchical structure: Category → SubCategory → Plans

Method	Endpoint	Auth	Description
GET	`/api/category`	Public	All categories with subcategories and plans (nested)
POST	`/api/category/add-subcategory`	Admin	Add new subcategory
POST	`/api/category/add-plan`	Admin	Add new plan under subcategory

🔢 **Frequency values:** `0` = Quarterly · `1` = HalfYearly · `2` = Yearly

ADD PLAN — Request Body

```
{
  "planName": "Farm Shield Basic",  "baseCoverageAmount": 100000,
  "coverageRate": 0.004,  "basePremium": 400,  "agentCommission": 50,
  "frequency": 2,  "subCategoryId": 5
}
```

🏠 6. Property Plans `/api/propertyplans`

Create and retrieve property insurance plans

Method	Endpoint	Auth	Description
GET	/api/propertyplans	Public	All property plans (flat list)
GET	/api/propertyplans/{id}	Public	Single plan by ID
POST	/api/propertyplans	Admin	Create new property plan

CREATE PLAN — Request Body

```
{
  "planName": "Luxury Penthouse Guard",  "baseCoverageAmount": 2000000,
  "coverageRate": 0.007,  "basePremium": 14000,  "agentCommission": 1000,
  "frequency": 0,  "subCategoryId": 1
}
```

🏠 Property Insurance System

🧾 7. Invoices

/api/invoices

GET /customer/invoices — Customer

Customer retrieves their own invoices with full plan and payment details.

RESPONSE FIELDS

Field	Description
invoiceNumber	e.g. INV-a3f2b1c0
generatedDate	ISO 8601 timestamp
planName	Insurance plan name
totalPremium	Total premium in ₹
installmentAmount	Per-installment amount
claimAmount	Claim payout amount

🔔 8. Notifications

/api/notifications

Method	Endpoint	Description
GET	/api/notifications	Latest 50 notifications
PUT	/{id}/read	Mark one as read
DELETE	/clear	Delete all notifications

NOTIFICATION OBJECT

```
{
  "id": 1,
  "title": "Agent Assigned",
  "message": "An agent has been assigned...",
  "type": "success",
  "isRead": false,
  "createdAt": "2026-03-03T12:00:00Z"
}
```

All endpoints require JWT. Any role can access their own notifications.

⚠️ Error Reference

Status	Meaning	Common Cause
200 OK	Request succeeded	Normal response
400 Bad Request	Invalid input	Validation error or business rule violation
401 Unauthorized	Missing/invalid JWT	Token expired or not provided
403 Forbidden	Insufficient role	Wrong role for this endpoint
404 Not Found	Resource missing	Invalid ID provided
500 Server Error	Unhandled error	Internal server exception

🔑 Test Credentials

Role	Email	Password
Admin	admin@gmail.com	Admin@123
Claims Officer	claims@gmail.com	Claims@123
Customer	customer@gmail.com	Customer@123

📊 API Summary

Controller	Base Route	Endpoints	Auth
AuthController	`/api/auth`	**4**	Partial
AdminController	`/api/admin`	**6**	Admin only
PolicyRequestsController	`/api/policyrequests`	**10**	Role-specific
ClaimsController	`/api/claims`	**4**	Role-specific
CategoryController	`/api/category`	**3**	Partial
PropertyPlansController	`/api/propertyplans`	**3**	Partial
InvoicesController	`/api/invoices`	**1**	Customer
NotificationsController	`/api/notifications`	**3**	Any (JWT)
Total		**34**	